Smart Campus Sensor & Room Management API

5COSC022W — Client-Server Architectures Coursework Report

This report contains the conceptual answers to all questions posed in each part of the coursework specification. Code, test cases, and introductory material are excluded as per the submission guidelines.

Part 1 — Service Architecture & Setup

Question: 1.1 — In your report, explain the default lifecycle of a JAX-RS Resource class. Is a new instance instantiated for every incoming request, or does the runtime treat it as a singleton? Elaborate on how this architectural decision impacts the way you manage and synchronize your in-memory data structures (maps/lists) to prevent data loss or race conditions

By default, JAX-RS creates a **new instance of every resource class for each incoming HTTP request** (Request-Scoped lifecycle). This means any instance-level fields initialised in a resource constructor are destroyed the moment the request completes — they cannot act as shared state across requests.

This has a direct impact on in-memory data management. If data were stored as instance fields (e.g., private Map<String, Room> rooms = new HashMap<>()), every request would start with a completely empty map, making persistent storage impossible.

To solve this, the project uses a **Singleton DataStore** class — instantiated exactly once by the JVM class loader using the eager singleton pattern. All resource classes call DataStore.getInstance() to access the shared maps. Because the same instance is always returned, all request-scoped resource instances always operate on the same underlying data.

Thread safety is achieved using **ConcurrentHashMap** instead of HashMap. When multiple clients send requests simultaneously, their threads all operate on the same map instance. ConcurrentHashMap partitions its internal buckets, allowing concurrent reads without locking and serialising only conflicting writes — preventing race conditions and data corruption without requiring explicit synchronized blocks.

Question: 1.2 — Why is the provision of ”Hypermedia” (links and navigation within responses) considered a hallmark of advanced RESTful design (HATEOAS)? How does this approach benefit client developers compared to static documentation?

HATEOAS (Hypermedia As The EngineOf ApplicationState) istheprinciplethatAPI responses should embed navigational links, enabling clients to discover available actions dynamically rather than relying on out-of-date static documentation.

- Self-documenting API: A client hitting GET /api/v1 receives a map of all available resource URLs. They do not need to know these paths in advance.
- Decoupled from URL structure: If the server restructures paths (e.g. v1 to v2), clients following links from responses adapt automatically without code changes.
- Reduced hardcoding: Clients that follow links are resilient to server-side restructuring, whereas clients with hardcoded URLs break immediately.
- Discoverability: New endpoints added to the API appear in the discovery response immediately — clients can use them without consulting updated documentation.

Part 2 — Room Management

Question: 2.1 — When returning a list of rooms, what are the implications of returning only IDs versus returning the full room objects? Consider network bandwidth and client side processing

Returning **IDs only** produces a minimal payload and reduces bandwidth, but forces the client to make N additional GET requests to retrieve details for each room — the N+1 request problem. This is appropriate when the client only needs to display a count or a summary list, and most items will never be clicked (lazy loading pattern).

Returning **full objects** requires a single round-trip and gives the client everything it needs immediately — suitable when a full table of room names, capacities, and sensor counts must be displayed at once. The trade-off is a larger payload, which may be significant if there are hundreds of rooms with many fields each.
This API returns full objects by default, as campus facilities managers typically need room names and capacities at a glance without making additional requests per room.

Question: 2.2 — Is the DELETE operation idempotent in your implementation? Provide a detailed justification by describing what happens if a client mistakenly sends the exact same DELETE request for a room multiple times

Strictly speaking, the implementation is **not idempotent in response code**, but it is **idempotent in server state** — which is the correct REST definition.

The first DELETE on room ENG-204 succeeds and returns 204 No Content. A subsequent identical DELETE returns 404 Not Found because the room no longer exists. The response code differs, but the server state after both calls is identical: the room does not exist.

REST's idempotency guarantee, as defined in RFC 7231, concerns the **effect on server state**, not the HTTP status code returned. The specification explicitly states that a method is idempotent if repeated identical requests have the same effect on the server. Therefore this implementation is correct REST behaviour — repeated DELETE calls never corrupt data or cause unexpected side effects.

Part 3 — Sensor Operations & Linking

Question: 3.1 — Weexplicitly use the @Consumes (MediaType.APPLICATION_JSON) annotation on the POST method. Explain the technical consequences if a client attempts to send data in a different format, such as text/plain or application/xml. How does JAX-RS handle this mismatch?

JAX-RS inspectsthe Content-Type header oftheincoming requestand comparesitagainstthe @Consumes annotation on the matching resource method. If they do not match, JAX-RS rejects the request **before the resource method is ever invoked** and automatically returns **HTTP 415 Unsupported Media Type**.

No custom error handling code is required — this enforcement is built into the JAX-RS runtime. The client receives a clear signal that the format of their request body is incompatible with what the server accepts. This enforces a strict contract: the API exclusively speaks JSON, and any deviation is cleanly rejected at the framework layer before any business logic runs.

Question: 3.2 —You implemented this filtering using @QueryParam. Contrast this with an alternative design where the type is part of the URL path (e.g.,/api/vl/sensors/type/CO2). Why is the query parameter approach generally considered superior for filtering and searching collections?

Path parameters identify**specific,discreteresources** (e.g. /sensors/TEMP-001). Query parameters **modify or filter a collection operation**. Using a path segment for filtering violates REST semantics — it implies /sensors/type/CO2 is itself a distinct resource, not a filtered view of the sensors collection.

- Optional by nature: ?type= can be omitted to return all sensors. A path segment is always required, making 'return all' awkward to express.
- Composability: Multiple filters compose naturally — ?type=CO2&status;=ACTIVE — whereas path segments become unreadable: /type/CO2/status/ACTIVE.
- REST convention: The REST architectural style reserves path segments for resource identification, not collection filtering.
- Caching: CDNs and proxies treat path-segment URLs as distinct resources, potentially caching filtered views incorrectly. Query parameters signal parameterised collection views to caching infrastructure.

Part 4 — Deep Nesting with Sub-Resources

Question: 4.1 — Discuss the architectural benefits of the Sub-Resource Locator pattern. How doesdelegating logic to separate classes help manage complexity in large APIs compared to defining every nested path (e.g., sensors/{id}/readings/{rid}) in one massive con troller class?

Without sub-resourcelocators, all nested paths — including /sensors/{id}/readings/{rid} — must be defined in one resource class. As the API grows, this leads to a monolithic controller handling sensor CRUD, reading history, filtering, and state management all in a single file. This violates the Single Responsibility Principle and makes the codebase increasingly difficult to test, maintain, and extend.

The Sub-Resource Locator pattern delegates handling of a URL subtree to a dedicated class. In this project, SensorResource handles /sensors and its direct children. When a request arrives at /sensors/{id}/readings, SensorResource returns a SensorReadingResource instance — JAX-RS then invokes the appropriate method on that instance. SensorReadingResource is entirely focused on reading history and has no knowledge of sensor registration or filtering.

- Separation of concerns: each class has one responsibility — smaller, more readable, more maintainable files.
- Independent testability: SensorReadingResource can be unit-tested in isolation without bootstrapping the entire sensor subsystem.
- Scalability: new sub-resources can be added without modifying the parent class, adhering to the Open/Closed Principle.
- Reusability: a sub-resource class could theoretically be reused under a different parent path with minimal changes.

Part 5 — Advanced Error Handling, Exception Mapping & Logging

Question: 5.2 — Why is HTTP 422 often considered more semantically accurate than a standard 404 when the issue is a missing reference inside a valid JSON payload?

404 Not Found communicates that the URL endpoint requested does not exist on the server — the client navigated to a location that was not found.

In the case of POST /api/v1/sensors with a non-existent roomId, the URL /api/v1/sensors exists and was found — this is not a 404 situation. The request body is syntactically valid JSON — this is not a 400 Bad Request. The semantic problem is that the payload contains a **broken reference**: the roomId field points to a room that does not exist in the system.

422 Unprocessable Entity (RFC 4918) was designed precisely for this case — the server understands the request format but cannot process the semantic content of the payload. It communicates: 'I received valid JSON, I understood your intent, but the data inside makes it impossible to fulfil this request.' This gives client developers far more actionable feedback than a generic 404, which would mislead the client into thinking the endpoint itself was missing.

Question: 5.4 — From a cybersecurity standpoint, explain the risks associated with exposing internal Java stack traces to external API consumers. What specific information could an attacker gather from such a trace?

A raw stack trace is effectively an attacker's reconnaissance report. It exposes several categories of sensitive information:

- Framework and library versions (e.g. jersey-server-2.40.jar) — the attacker can look up known CVEs for that exact version and craft targeted exploits.
- Internal package and class names (e.g. com.smartcampus.store.DataStore) — reveals the project's internal architecture, naming conventions, and design decisions.
- Source file names and line numbers (e.g. DataStore.java:87) — pinpoints exactly where to look for vulnerabilities in leaked or decompiled code.

- Business logic clues — method names like validateRoomOwnership or calculateBillingRate reveal domain logic that attackers can attempt to bypass.
- Server infrastructure details — OS paths, JVM version, and deployment structure that help fingerprint the server environment for further attacks.

The GlobalExceptionMapper in this project ensures no Throwable ever reaches the client as a stack trace. All unhandled exceptions are logged server-side — where only authorised engineers can access them — and the client receives only a generic, information-free 500 Internal Server Error JSON body.

Question: 5.5 — Why is it advantageous to use JAX-RS filters for cross-cutting concerns like logging, rather than manually inserting Logger.info() statements inside every single re source method?

Manual logging requires inserting Logger.info() at the start and end of every resource method. With 15+ methods across four resource classes, this leads to severe maintainability problems:

- Code duplication: 30+ nearly identical log statements spread across the codebase.
- Inconsistency: each developer formats log entries differently, making log analysis unreliable.
- High maintenance cost: changing the log format requires editing every method.
- Violation of SRP: business logic becomes interleaved with infrastructure concerns.
- Error-prone: new endpoints added without manual logging go completely unobserved.

A single filter class implementing both ContainerRequestFilter and ContainerResponseFilter intercepts every request and response automatically. Logging is **guaranteed for all endpoints** — including new ones added in the future — with zero changes to resource classes. This is the standard industry approach to cross-cutting concerns (logging, authentication, CORS, rate limiting), and it is the primary reason JAX-RS filters exist as a feature.